

Laboraufgabe: Automatisierung eines Rollltors via OPC UA

Praktikum Steuerungstechnik mit Siemens S7-1500

Zielsetzung der Laborübung

In dieser Laborübung programmieren Sie die Steuerung für eine Rollltor-Simulation. Die physikalische Simulation des Tors läuft als eigenständiges Python-Programm und kommuniziert über das standardisierte Protokoll **OPC UA** mit einer simulierten Speicherprogrammierbaren Steuerung (Siemens S7-1500 über PLCSIM Advanced).

Lernziele dieser Übung sind:

- Einrichtung eines OPC UA Servers auf einer S7-1500.
- Erstellung von Datenaustausch-Schnittstellen (Datenbausteinen).
- Programmierung grundlegender Verriegelungen und Selbsthaltungen in KOP und FUP.
- Fehlerbehandlung und Ablaufsteuerungen in SCL.

Wichtiger Hinweis zur Simulation

Die Rollltor-Simulation simuliert reale physikalische Grenzen. Wird ein Motor nicht rechtzeitig gestoppt und überfährt das Tor die Endlagen ($> 102\%$ oder $< -2\%$), geht das System in einen kritischen Fehlerzustand (Overtravel Fault). Gleiches gilt, wenn beide Motoren (Auf/Ab) gleichzeitig angesteuert werden.

1 Phase 1: Infrastruktur und Setup

Bevor Sie mit der eigentlichen Programmierung beginnen können, muss die Kommunikationsschnittstelle zwischen dem TIA Portal (SPS) und der Simulation (Python) eingerichtet werden.

1.1 Aufgabe 1.1: Datenbaustein (DB) anlegen

Legen Sie im TIA Portal einen neuen globalen Datenbaustein an (z.B. `DB_Rolltor OPC`). Dieser dient als Schnittstelle. Legen Sie darin exakt die folgenden Variablen an:

Variablenname	Datentyp	Beschreibung / Richtung	Erreichbar (OPC)
Position	Real	Aktuelle Position des Tors (0.0 bis 100.0) [Eingang]	Ja
Error	Bool	Störsignal der Simulation (True = Fehler) [Eingang]	Ja
Upper_Limit	Bool	Oberer Endschalter (100 %) erreicht [Eingang]	Ja
Lower_Limit	Bool	Unterer Endschalter (0 %) erreicht [Eingang]	Ja
Motor_Up	Bool	Steuerbefehl: Motor AUF [Ausgang]	Ja
Motor_Down	Bool	Steuerbefehl: Motor AB [Ausgang]	Ja

1.2 Aufgabe 1.2: OPC UA Server aktivieren

1. Öffnen Sie die Gerätekonfiguration der S7-1500.
2. Navigieren Sie zu *OPC UA* → *Server* und aktivieren Sie den OPC UA Server.
3. Notieren Sie sich die Server-URL (z.B. `opc.tcp://192.168.0.1:4840`).
4. Stellen Sie sicher, dass in den Eigenschaften des erstellten Datenbausteins die Variablen für OPC UA freigegeben sind (Spalte: *Erreichbar aus HMI/OPC UA*).
5. Übersetzen Sie das Projekt und laden Sie es in die simulierte Steuerung (PLCSIM Advanced).

1.3 Aufgabe 1.3: Verbindungstest

1. Verbinden Sie sich zunächst mit einem Diagnose-Tool (z.B. *UaExpert*) auf die SPS und prüfen Sie, ob Sie die Variablen im Adressraum sehen und lesen/schreiben können.
2. Tragen Sie die korrekten Node-IDs (z.B. `ns=3;s="DB_Rolltor OPC"."Motor_Up"`) und die Server-URL in die `config.json` der Python-Simulation ein.
3. Starten Sie die Simulation im `client`-Modus und prüfen Sie, ob sich die Werte bei Betätigung der GUI-Buttons im TIA Portal ändern.

2 Phase 2: Programmieraufgaben (IEC 61131-3)

Lösen Sie die folgenden Aufgaben in den angegebenen Programmiersprachen. Kapseln Sie jede Aufgabe sinnvoll in eigenen Funktionen (FC) oder Funktionsbausteinen (FB) und rufen Sie diese im **OB1** (Main) auf.

2.1 Aufgabe 2.1: Totmannsteuerung (Tippbetrieb)

Sprache: Kontaktplan (KOP)

Szenario: Das Tor soll manuell gesteuert werden können, solange Taster gedrückt gehalten werden.

- Erstellen Sie zwei Merker/Eingänge in der SPS für virtuelle Taster: **Taster_Hand_Auf** und **Taster_Hand_Ab**.
- Das Tor soll nur fahren, solange der jeweilige Taster ein **True**-Signal liefert.
- **Verriegelung:** Es muss hardwaretechnisch ausgeschlossen sein, dass **Motor_Up** und **Motor_Down** gleichzeitig aktiv sind.
- **Endlagen-Stopp:** Wird der Endschalter (**Upper_Limit** bzw. **Lower_Limit**) erreicht, muss der jeweilige Motor zwingend abschalten, auch wenn der Taster noch gedrückt ist.

2.2 Aufgabe 2.2: Automatikbetrieb mit Selbsthaltung

Sprache: Funktionsplan (FUP)

Szenario: Das Tor soll nach einem kurzen Tastendruck automatisch bis zur Endlage fahren.

- Nutzen Sie SR-Flipflops (Setzen/Rücksetzen), um den Motor-Zustand zu speichern.
- Ein kurzer Impuls auf **Taster_Auto_Auf** soll das Flipflop für **Motor_Up** setzen.
- **Wichtig:** Das Rücksetzen (Reset) des Flipflops muss durch den Endschalter (**Upper_Limit**) erfolgen.
- Eine gegenseitige Verriegelung ist auch hier zwingend erforderlich!
- *Achtung:* Wenn das Rücksetzen fehlschlägt, fährt die Simulation das Tor über 100 % hinaus in den mechanischen Crash (Overtravel Fault).

2.3 Aufgabe 2.3: Störungsbehandlung und Diagnose

Sprache: Structured Control Language (SCL)

Szenario: Die Anlage soll sicher auf Fehler reagieren und sich ordnungsgemäß quittieren lassen.

- Wenn das Signal **Error** von der Simulation auf **True** geht, müssen alle Fahrbefehle (**Motor_Up**, **Motor_Down**) **sofort** überschrieben und auf **False** gesetzt werden.
- Dieser Störungs-Stopp hat die höchste Priorität im Programm.
- Implementieren Sie einen virtuellen Taster **Taster_Reset**.
- Eine erneute Fahrt darf erst wieder freigegeben werden, wenn **Error = False** ist und der Fehler über **Taster_Reset** (positive Flanke) quittiert wurde.

2.4 Aufgabe 2.4: Automatische Zufahrt (Ablaufsteuerung)

Sprache: SCL oder S7-GRAH

Szenario: Das Tor soll im Automatikbetrieb nach einer Wartezeit selbstständig schließen.

- Das Tor wird geöffnet. Sobald der obere Endscharter (**Upper_Limit**) erreicht wird, soll eine Timer-Funktion (Einschaltverzögerung TON) gestartet werden.
- Die Wartezeit beträgt 10 Sekunden.
- Nach Ablauf der 10 Sekunden schließt sich das Tor automatisch (Setzen von **Motor_Down**).
- **Sicherheitsabbruch:** Wird während der Schließfahrt (oder während der Wartezeit) der Taster **Taster_Auto_Auf** gedrückt, muss die Zufahrt abgebrochen und das Tor sofort wieder geöffnet werden.

Abgabe

Senden Sie am Ende des Praktikums das archivierte TIA-Projekt (.zap16/.zap17) sowie ein kurzes PDF-Dokument mit Screenshots Ihrer Code-Bausteine und einer kurzen Beschreibung Ihrer Verriegelungs-Logik an den Betreuer.